

AL2002 Artificial Intelligence Lab Manual
ver 2.0

Saad Ahmad, BS

Inst., Dept. of Computer Science

National University of Computer & Emerging Sciences

Artificial Intelligence Lab Manual

Saad Ahmad

Instructor, Dept. of Computer Science

National University of Computer & Emerging Sciences

160 Industrial Estate, Peshawar, KPK, Pakistan

Version History

1.0 Spring 2026: Compilation of all previous labs in book format with minor updates & corrections {**Saad Ahmad**}

2.0 Spring 2026: Restructured manual: updated chapter titles, added Simulated Annealing, new CSP chapter (Lab 9), Linear & Logistic Regression in Ch. 13, improved heuristics explanation, added TikZ figures & code snippets {**Saad Ahmad**}

This manual is customized for course AL2002 Artificial Intelligence taught at the National University of Computer & Emerging Sciences, Peshawar, Pakistan. The lab portion of this course provides cover for 1 credit hour. Evaluation for this lab is at discretion of your instructor and/or teacher.

The manual can also be used as a learning tool for other artificial intelligence courses both at undergraduate or graduate level. Each chapter in this manual is intended to serve as a separate lab, although instructors should note that some chapter's require more than one lab sessions to complete.

Contents

I	MACHINE LEARNING	5
1	Unsupervised Learning: K-Means Clustering	7
1.1	What is Unsupervised Learning?	7
1.2	What is Clustering?	7
1.3	The K-Means Algorithm	7
1.3.1	How It Works: Step-by-Step	7
1.4	Distance and Centroids	8
1.4.1	1. Euclidean Distance	8
1.4.2	2. Calculating the Centroid (Mean)	8
1.5	Python Implementation: Building K-Means from Scratch	9
1.5.1	Part 1: The Dataset & Distance Function	9
1.5.2	Part 2: Finding the Closest Centroid	9
1.5.3	Part 3: Updating the Centroids	10
1.5.4	Part 4: The Complete Training Loop	10
1.6	How to Choose K? (The Elbow Method)	11
1.7	Lab Tasks: K-Means in Practice	12
1.7.1	Task 1: Calculate the Error (SSE)	12
1.7.2	Task 2: Predict New Data	12
1.7.3	Task 3: Challenge - Handling 3D Data	13

Part I

MACHINE LEARNING

Chapter 1

Unsupervised Learning: K-Means Clustering

1.1 What is Unsupervised Learning?

In our previous labs, we looked at Supervised Learning (like KNN and Naive Bayes), where we had access to the correct "answers" or labels. But what happens when you don't have labeled data?

Imagine someone dumps a massive box of assorted LEGO bricks on the floor and asks you to "organize" them. They don't give you a manual telling you what to put where; they just want the chaos to make sense. You might decide to group them by color, or maybe by size, or perhaps by shape.

This is **Unsupervised Learning**. We feed the computer raw, unorganized data, and ask it to find the hidden patterns, groups, or rules on its own.

1.2 What is Clustering?

Clustering is the most common unsupervised learning task. It involves taking a dataset and dividing it into subgroups (clusters) such that data points in the same group are very similar to each other, and points in different groups are very different.

Think of a supermarket. They use clustering to arrange items in aisles. Dairy products go in one corner, fresh produce in another, and cleaning supplies in a different section. This makes it easier for you to find things because items with similar properties are grouped together!

1.3 The K-Means Algorithm

K-Means is a popular, simple, and fast clustering algorithm. The "K" stands for the number of clusters you want the algorithm to find. A "Mean" or "Centroid" is the center point of a cluster.

The goal of K-Means is to position K centroids so that the distance from each point to its nearest centroid is as small as possible.

1.3.1 How It Works: Step-by-Step

1. **Initialization:** Choose the number of clusters, K . Randomly pick K data points to act as the starting centroids.

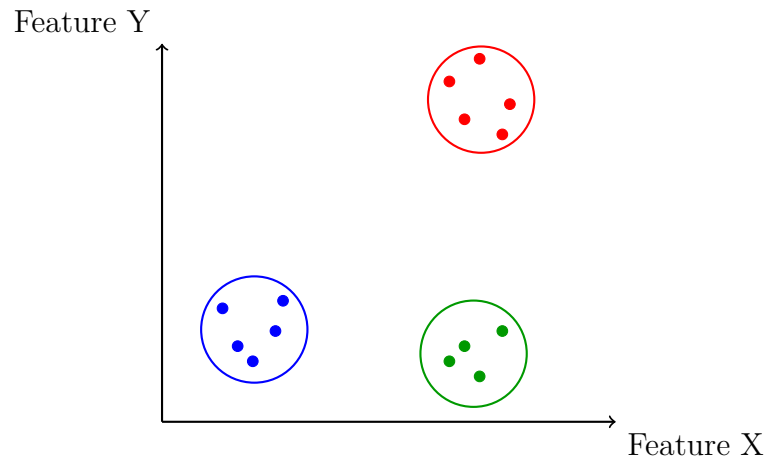


Figure 1.1: Clustering groups similar points together based on their features.

2. **Assignment Step:** For every single point in the dataset, calculate the distance to all K centroids. Assign the point to the cluster of the centroid it is closest to.
3. **Update Step:** Now that the points have been grouped, calculate the new "mean" (average) position of all points in each cluster. Move the centroid to this new mean position.
4. **Repeat:** Repeat Steps 2 and 3 until the centroids stop moving (convergence).

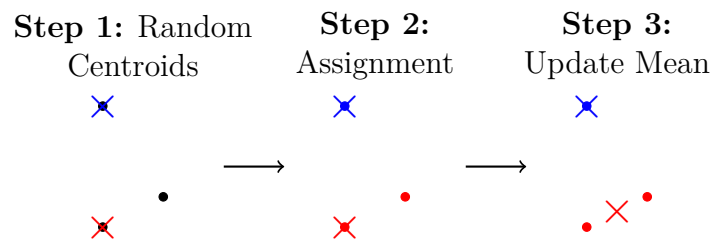


Figure 1.2: The iterative process of K-Means.

1.4 Distance and Centroids

Before we dive into the code, let's look at the math.

1.4.1 1. Euclidean Distance

Just like in KNN, K-Means uses Euclidean distance to figure out which centroid is closest. For a 2D point, the formula is:

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

1.4.2 2. Calculating the Centroid (Mean)

When a cluster has several points, we need to find their exact center to move the centroid there. We do this by calculating the average of all the X coordinates and the average of all the Y coordinates.

Checkpoint 1: Manual Centroid Update

Imagine a red cluster with three points: $A(2, 4)$, $B(4, 6)$, and $C(6, 2)$.

- Mean X: $(2 + 4 + 6)/3 = 12/3 = 4$
- Mean Y: $(4 + 6 + 2)/3 = 12/3 = 4$

Question: If a blue cluster has points $P(10, 20)$ and $Q(20, 20)$, what is the new X and Y coordinate of the blue centroid?

1.5 Python Implementation: Building K-Means from Scratch

Let's build K-Means step by step! We will use simple lists to keep the logic clear.

1.5.1 Part 1: The Dataset & Distance Function

We start with some simple points and our Euclidean distance function (from last lab).

```

1 import math
2 import random
3
4 # A simple 2D dataset.
5 # We can clearly see two groups visually:
6 # (1,1), (1,2) vs (10,10), (10,11)
7 dataset = [
8     [1, 1], [1, 2], [2, 1],
9     [10, 10], [10, 11], [11, 10]
10 ]
11
12 def euclidean_distance(point1, point2):
13     total = 0
14     for i in range(len(point1)):
15         diff = point1[i] - point2[i]
16         total = total + (diff * diff)
17     return math.sqrt(total)

```

1.5.2 Part 2: Finding the Closest Centroid

For every data point, we must find out which centroid is closer to it.

```

1 def get_closest_centroid(point, centroids):
2     # Start with a very large distance (infinity) so the first distance we
3     # check will always be smaller
4     min_dist = float('inf')
5     closest_index = -1
6
7     for i in range(len(centroids)):
8         dist = euclidean_distance(point, centroids[i])
9         if dist < min_dist:
10             min_dist = dist
11             closest_index = i
12
13     return closest_index

```

What is happening? We loop through all available centroids. We initialize `min_dist` to `float('inf')`, which represents "infinity" in Python. We do this to guarantee that the very first distance we calculate will be smaller than `min_dist`, allowing us to properly track the smallest distance found. If we find a distance smaller than our current minimum, we remember that centroid's index.

1.5.3 Part 3: Updating the Centroids

Once the points are assigned to groups, we calculate the new mean position.

```

1 def update_centroids(clusters):
2     new_centroids = []
3
4     for cluster in clusters:
5         # If a cluster randomly ended up empty, keep it empty to avoid divide-
6         # by-zero
7         if len(cluster) == 0:
8             continue
9
10        num_points = len(cluster)
11
12        # Calculate the average across all dimensions (X, Y)
13        new_centroid = []
14        for dim in range(len(cluster[0])):
15            dim_sum = 0
16            for point in cluster:
17                dim_sum += point[dim]
18            new_centroid.append(dim_sum / num_points)
19
20        new_centroids.append(new_centroid)
21
22    return new_centroids

```

1.5.4 Part 4: The Complete Training Loop

Now we tie it all together.

```

1 def k_means(dataset, k, max_iterations=10):
2     # Step 1: Initialize random centroids
3     # Pick the first 'k' points as a starting point (simple approach)
4     centroids = dataset[:k]
5
6     for iteration in range(max_iterations):
7         print(f"--- Iteration {iteration+1} ---")
8
9         # Create empty lists for our k clusters
10        clusters = []
11        for _ in range(k):
12            clusters.append([])
13
14        # Step 2: Assign points
15        for point in dataset:
16            centroid_index = get_closest_centroid(point, centroids)
17            clusters[centroid_index].append(point)
18
19        # Step 3: Update centroids
20        new_centroids = update_centroids(clusters)
21

```

```

22     # Check if they stopped moving
23     if new_centroids == centroids:
24         print("Centroids stopped moving! Convergence reached.")
25         break
26
27     centroids = new_centroids
28     print("Centroids updated to:", centroids)
29
30     return centroids, clusters
31
32 # Run the algorithm
33 k = 2
34 final_centroids, final_clusters = k_means(dataset, k)
35
36 for i in range(len(final_clusters)):
37     print(f"Cluster {i+1} points:", final_clusters[i])

```

Student Activity 1: Run the complete K-Means code.

1. Add a completely random point to the dataset like [5, 5] (right in the middle). Which cluster does it eventually join?
2. Change K to 3. What happens to the clusters? Does it still make sense for this specific data?

1.6 How to Choose K? (The Elbow Method)

In our simple example, it was obvious that $K = 2$ because we could see the two groups. But what if there are 10,000 points spanning 5 dimensions? You can't visualize that.

To find the best K , we use the **Elbow Method**. We run K-Means using $K = 1, 2, 3, 4, 5...$ and each time we calculate the **Sum of Squared Errors (SSE)**. SSE measures the total squared distance between every point and its assigned centroid. The mathematical formula is:

$$\text{SSE} = \sum_{i=1}^K \sum_{x \in C_i} \text{distance}(x, \mu_i)^2$$

Where:

- K is the total number of clusters.
- C_i is the i -th cluster.
- x is a data point inside that cluster.
- μ_i is the centroid of that cluster.

As K increases, the SSE will always drop because points get closer to their centroids. However, there is a "sweet spot" where the error stops dropping rapidly and levels out. This spot looks like the bend of an elbow.

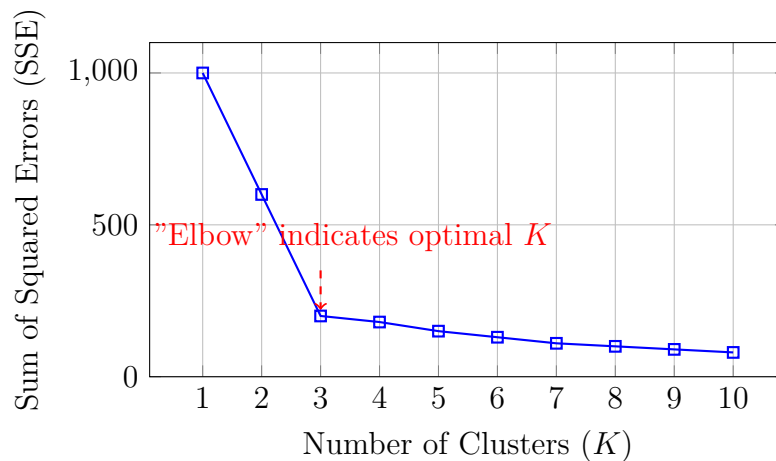


Figure 1.3: The Elbow Method showing an optimal $K = 3$.

1.7 Lab Tasks: K-Means in Practice

Now that we have built the core engine of K-Means, it is time to write some code to test it in practical scenarios. Complete the following tasks by extending your Python script.

1.7.1 Task 1: Calculate the Error (SSE)

We know that the Sum of Squared Errors (SSE) tells us how good our clusters are. Write a Python function that takes your final `clusters` and `centroids` as input, and returns the total SSE.

```

1 def calculate_sse(clusters, centroids):
2     total_error = 0
3     # TODO: Loop through every cluster
4     # TODO: Loop through every point in that cluster
5     # TODO: Calculate the squared euclidean_distance between the point and the
        cluster's centroid
6     # TODO: Add it to total_error
7     return total_error

```

1.7.2 Task 2: Predict New Data

Once K-Means has finished training and you have your final centroids, you can use those centroids to classify new, unseen data points without having to retrain the whole model.

Write a function that accepts a new data point and the list of trained centroids, and returns the index of the cluster it belongs to. *(Hint: You already wrote a helper function earlier that does exactly this!)*

```

1 def predict(new_point, final_centroids):
2     # TODO: Return the index of the closest centroid
3     pass
4
5 # Test your code:
6 # print("Point [8, 8] belongs to cluster:", predict([8, 8], final_centroids))

```

1.7.3 Task 3: Challenge - Handling 3D Data

The dataset we worked with had 2 features (X and Y coordinates). What happens if our data becomes more complex?

Create a new small dataset containing **3D points** (e.g., `[[1, 2, 5], [2, 1, 4], [10, 8, 9], ...]`). Without changing the internal logic of your `euclidean_distance` or `update_centroids` functions, run your `k_means` algorithm on this new 3D data.

Reflection Question: Did your code crash, or did it work seamlessly? Explain why your custom Euclidean distance loop handles 3-dimensional (and 4D, 5D...) data automatically.

Bibliography

- [1] Agents in artificial intelligence.
- [2] Artificial intelligence.
- [3] Artificial intelligence.
- [4] Artificial intelligence - agents and environments.
- [5] Cs6-1 lab 06.
- [6] Networkx - python software package for complex networks.
- [7] Types of artificial intelligence.
- [8] What is intelligent agent in ai - types and function.